

Le Chiamate di Sistema

Lezione 10

Sistemi Operativi - Il Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

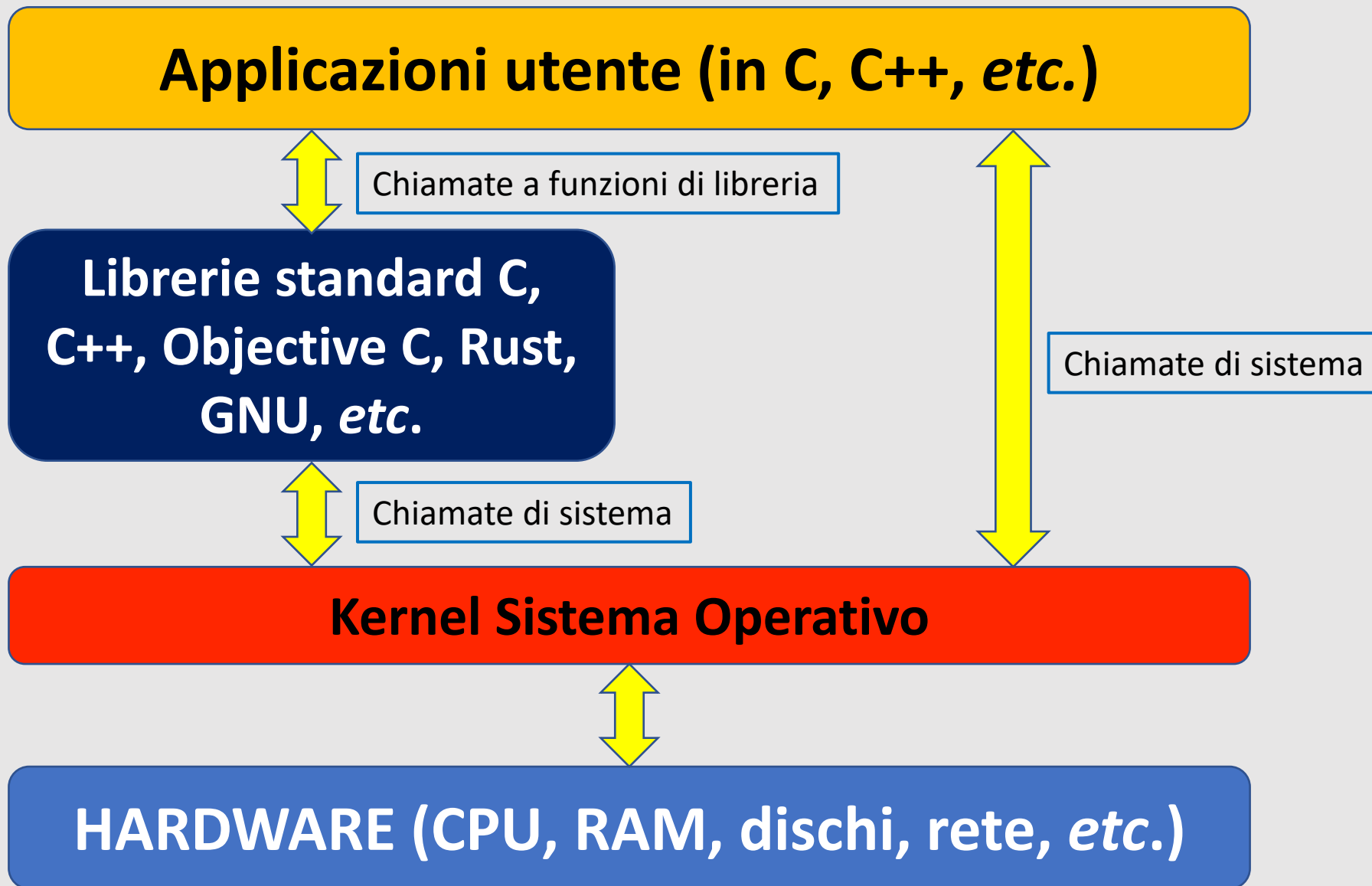
- Panoramica
- Introduzione alla programmazione di sistema
- System call memoria
 - (richiami)
- System call file I/O

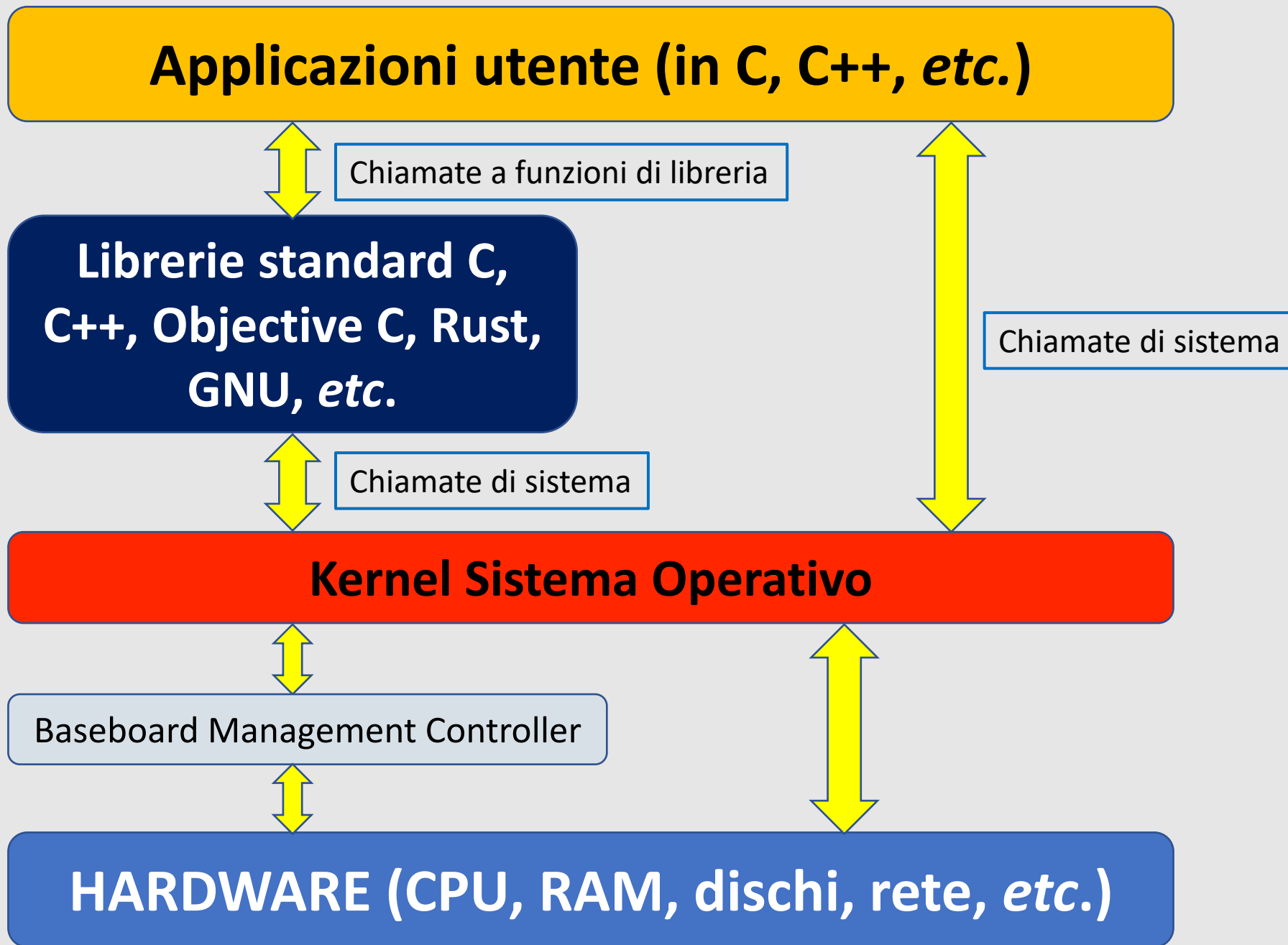


Programmazione di sistema

- Il Kernel
 - componente del sistema operativo che gestisce le risorse disponibili
 - offre l'accesso e l'utilizzo delle risorse da parte dei processi
- Le risorse principali gestite sono:
 - CPU Central Processing Unit
 - RAM Random Access Memory
 - I/O I dispositivi di Input e Output
- System call
 - Limitato numero di punti di accesso al kernel
 - Il numero varia con la versione del sistema: UnixV7 50, 4.4BSD 110, Linux 380, ...
 - Permettono ad un processo di accedere ai servizi offerti dal kernel
 - quindi ... permettono al programmatore di creare programmi che interagiscono direttamente con il kernel

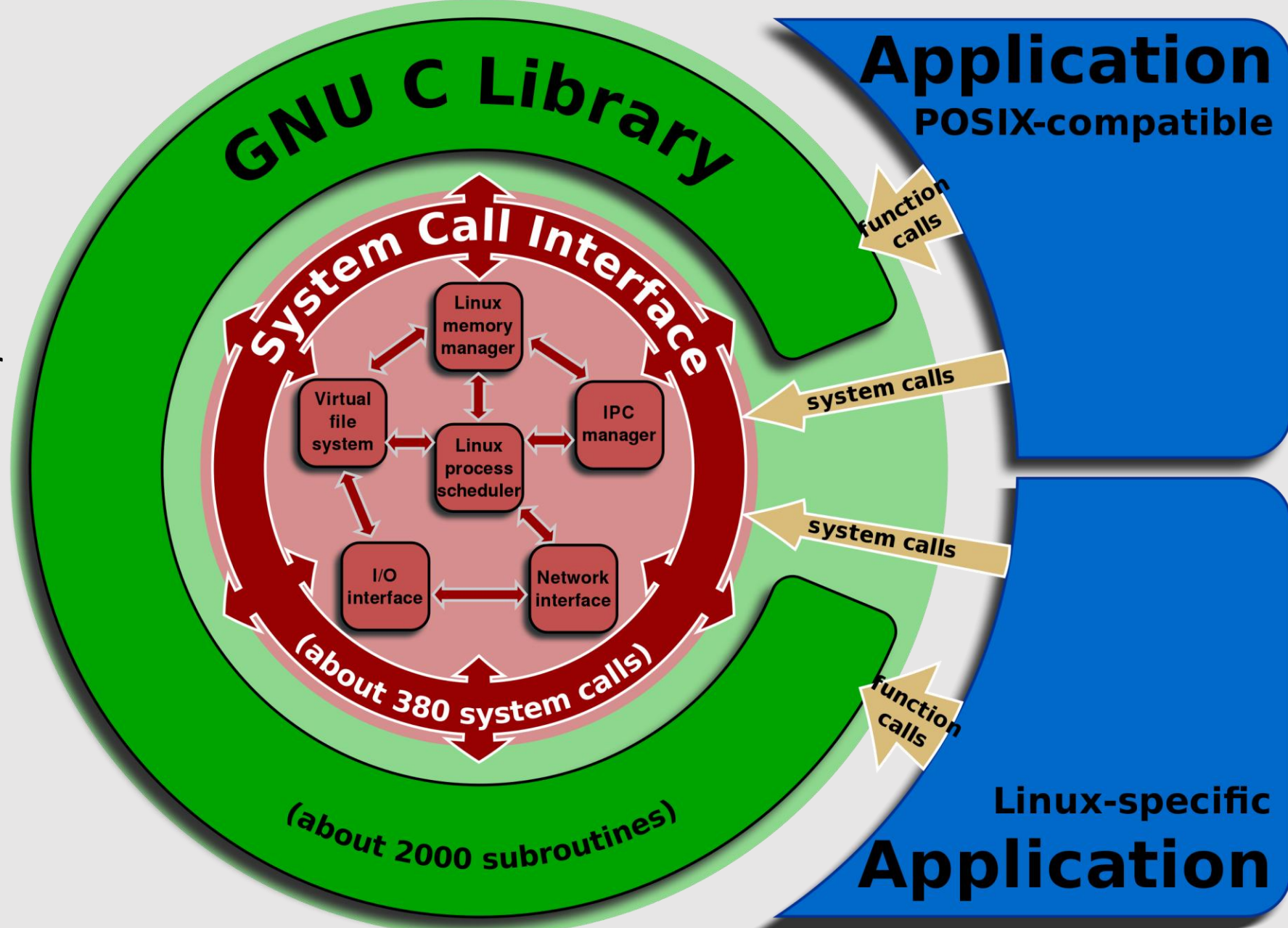






System call

interfaccia del Kernel per la programmazione



https://en.wikipedia.org/wiki/Linux_kernel_interfaces_-_/media/File:Linux_kernel_System_Call_Interface_and_glibc.svg



System call (cont.)

- Definizione delle system call è in linguaggio C indipendentemente dalla tecnica usata per invocare la system call
- Per ogni system call esiste una funzione C avente lo stesso nome
 - Il processo utente invoca questa funzione usando la sequenza standard C
 - Questa funzione invoca il corrispondente servizio del kernel utilizzando la tecnica necessaria
 - Ad esempio mettendo i parametri della funzione C in un registro e poi generando un interrupt per il kernel usando istruzioni macchina

Dove trovare la descrizione delle SysCall

- Sezione 2 del man
 - Informazioni dettagliate sull'utilizzo ed il funzionamento delle system call
 - `man 2 nome_system_call`

```
$ man man
```

La seguente tabella mostra i numeri delle sezioni del manuale seguiti dai tipi di pagine che contengono

1	Programmi eseguibili e comandi della shell
2	Chiamate al sistema (funzioni fornite dal kernel)
3	Chiamate alle librerie (funzioni all'interno delle librerie di sistema)
4	File speciali (di solito trovabili in <u>/dev</u>)
5	Formati dei file e convenzioni p.es. <u>/etc/passwd</u>
6	Giochi
7	Pacchetti di macro e convenzioni p.es. man (7), groff (7).
8	Comandi per l'amministrazione del sistema (solitamente solo per root)
9	Routine del kernel [Non standard]

Funzioni di libreria generali

- Non sono punti di accesso ai servizi del kernel
- Possono invocare una o più system call
 - Ad esempio **printf** può usare la system call **write** per scrivere una stringa in output
- ... ma anche nessuna
 - ad es: **strcpy**, **atoi**



Dove trovare la descrizione delle funzioni di libreria generali

- Sezione 3 del man
 - Informazioni dettagliate sull'utilizzo ed il funzionamento delle funzioni di libreria
 - `man 3 nome_funz_libreria`

```
$ man man
```

La seguente tabella mostra i numeri delle sezioni del manuale seguiti dai tipi di pagine che contengono

1	Programmi eseguibili e comandi della shell
2	Chiamate al sistema (funzioni fornite dal kernel)
3	Chiamate alle librerie (funzioni all'interno delle librerie di sistema)
4	File speciali (di solito trovabili in <u>/dev</u>)
5	Formati dei file e convenzioni p.es. <u>/etc/passwd</u>
6	Giochi
7	Pacchetti di macro e convenzioni p.es. man (7), groff (7).
8	Comandi per l'amministrazione del sistema (solitamente solo per root)
9	Routine del kernel [Non standard]

Differenza tra system call e funzioni generali

- Entrambe sono funzioni C
- Entrambe forniscono servizi ad un'applicazione
- **Una funzione generale può essere rimpiazzata ma una system call no!!!**
 - La syscall **sbrk** permette di allocare la memoria
 - La **malloc** è implementata usando la **sbrk**
 - Possiamo implementare la nostra **malloc** ma dovremo sempre utilizzare la **sbrk**
- Le system call introducono una separazione di compiti (o responsabilità)
 - La **sbrk** alloca porzioni di memoria (*kernel mode*) per il processo utente. La **malloc** gestisce l'area di memoria in *user mode*

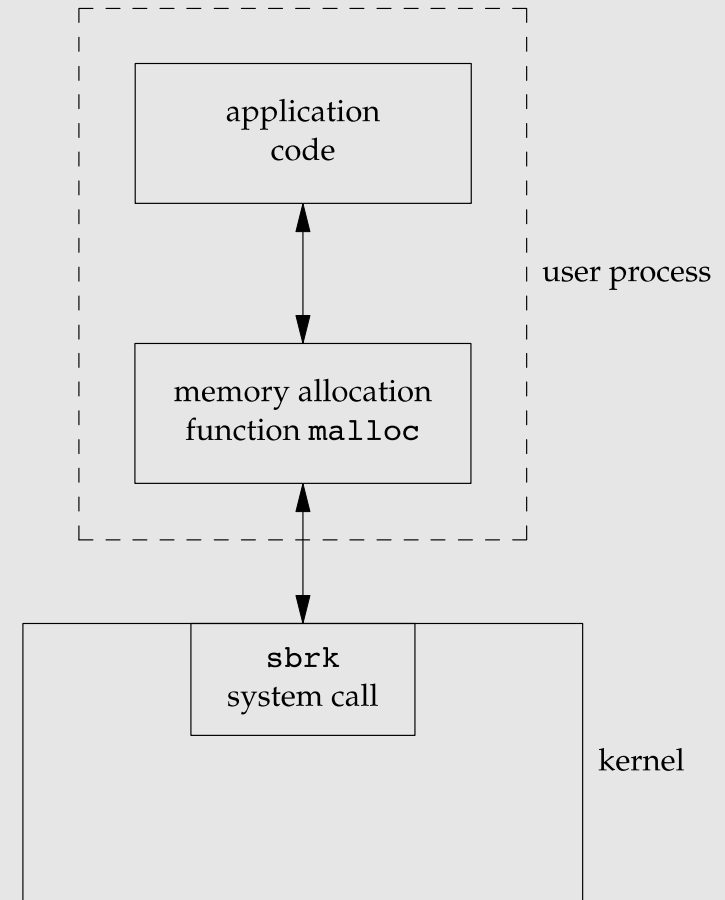
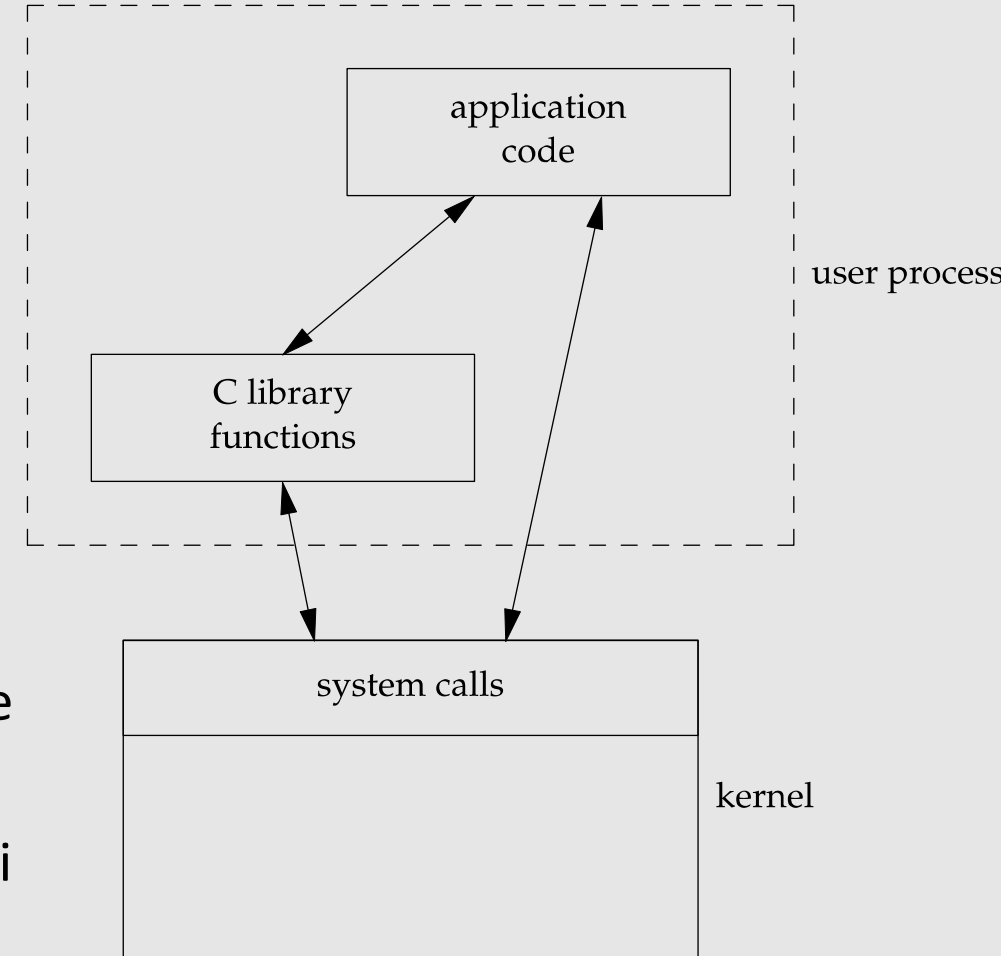


Figure 1.11 Separation of malloc function and sbrk system call

Differenza tra system call e funzioni general purpose

- Le funzioni di libreria semplificano l'uso delle system call
 - System call espongono un'interfaccia minimale
 - Le funzioni di libreria forniscono funzionalità elaborate e semplificano la gestione delle strutture dati di input e output
 - Maggiore portabilità delle applicazioni
- Ad esempio:
 - Le system call per la gestione dei processi sono solitamente invocate direttamente dall'applicazione
 - Da un processo possiamo invocare la syscall **time** per ottenere il tempo in secondi trascorso dal 1 gennaio 1970 ma possiamo anche invocare funzioni come la **ctime** o **asctime** per ottenere una stringa "Fri Apr 9 16:05:01 2021"



Le system call

- File
 - File e Directory
 - Inter Process Communication (IPC): pipe e fifo
 - Socket e Socket di rete (networking)
- Gestione della memoria
- Processi
 - Gestione di processi
 - Gestione dei thread (POSIX Thread)
 - Segnali



System call: Files

- Gestione dei file e directory
 - creazione
 - accesso/modifica contenuto
 - accesso/modifica attributi
- Comunicazione tra processi
 - *pipe* e *fifo*
 - comunicazione tra processi che risiedono sulla stessa macchina fisica/virtuale
 - *socket*
 - comunicazione tra processi mediante protocolli di rete (incluso IPV4/V6)



System call: Processi

- Gestione dei processi
 - creazione/terminazione, esecuzione,
 - sincronizzazione,
 - accesso/modifica attributi/ambiente
- Gestione dei segnali
 - interazione/comunicazione tra processi
 - creazione sezioni critiche
- POSIX Thread
 - processi cosiddetti *lightweight* (leggeri)
 - creazione/terminazione, sincronizzazione



System call: funzioni di libreria correlate

- Errori
 - perror
 - strerror
 - errno



Gestione errori system call

- L'esecuzione di una system call può interrompersi e non andare a buon fine per diversi motivi, tra cui principalmente:
 - il processo che la invoca non ha sufficienti privilegi per l'esecuzione
 - non ci sono sufficienti risorse per l'esecuzione
 - gli argomenti in ingresso alla system call non sono validi
- E' fondamentale controllare i valori di ritorno per rilevare e segnalare all'utente il verificarsi di errori.
- Per un corretto funzionamento del programma è FONDAMENTALE GESTIRE IN MANIERA OPPORTUNA l'eventuale errore verificatosi

errno

- Le system call che terminano con un errore
 - tipicamente ritornano il valore -1 (controllare caso per caso!!)
 - impostano la **variabile globale** `errno` con il codice specifico dell'errore che si è generato durante l'esecuzione
- Le system call che terminano con successo lasciano invariato `errno`
- La variabile `errno` rappresenta il codice di errore dell'ultima system call invocata che ha generato un errore
 - Ha senso utilizzare tale variabile solo dopo essersi accertati che la system call ha ritornato un valore di errore (-1). Altrimenti si rischia di considerare il valore di `errno` inconsistente.

perror ()

- funzione della libreria standard

```
#include <stdio.h>
```

```
void perror(const char *prefix);
```

- **stampa su** `stderr` il messaggio di errore convertendo il codice di errore `errno` in una stringa formata da

<prefix>:<errno_string>

- `<errno_string>` rappresenta il messaggio di errore in formato di stringa (e quindi mnemonico) associato al valore di `errno`.
- ad es. `perror("main");` invia su `stderr` la stringa
`"main:messaggio_errore_mnemonico=_errno"`

strerror()

- funzione di libreria che consente di convertire un codice di errore numerico `errno` che acquisisce come parametro di input nella sua equivalente rappresentazione in stringa.

```
#include <string.h>  
char *strerror(int errnum);
```

- ad esempio:

```
printf("Si è verificato errore:%s\n", strerror(errno));
```

Esempio gestione errore

```
..  
#include <syscall_lib.h>  
#include <stdio.h> /* per perror() */  
#include <string.h> /* per strerror() */  
#include <errno.h>  
..  
if (una-syscall() == -1) {  
    int errsv = errno;  
    perror("main"); // stderr  
    printf("Si è verificato errore:%s\n", strerror(errsv)); //stdout  
    if (errsv == ...) {  
        ...  
    }  
} ..
```

‘Disinfestare’ programmi usanti system calls

- È spesso utile monitorare il comportamento di un processo relativamente all’invocazione di system call.
- `strace` traccia l’invocazione di system call da parte di un processo.
- `strace` consente di stampare la lista di system call invocate da un processo e relativi parametri

```
$strace -o strace.txt -s <max_string_size> /path2/program
```

```
$strace -p <pid> -e trace=syscall1,syscall2... -o strace_<pid>.txt
```

```
$strace -o passwd.strace -s 100 cat /etc/passwd
```

```
$cat passwd.strace
```



Allocazione della memoria

```
#include <stdlib.h> /*funzioni di libreria, non sono  
                    system call */
```

```
/* allocano nell'heap (vedi Lezione 9) */  
void *malloc(size_t size);  
void *calloc(size_t nmemb, size_t size);  
void *realloc(void *ptr, size_t size);  
void free(void *ptr);
```

```
#include <alloca.h> /* alloca nello stack */  
void *alloca(size_t size);
```



mmap, brk, sbrk system call

- Le varie `m/c/re/alloc` usano le vere system call per la gestione della memoria, ad es.
 - `mmap` - 'mappa' file o dispositivi in memoria
 - `brk` - cambia dimensione data segment processo

```
void *mmap(void *addr, size_t length, int prot,  
           int flags, int fd, off_t offset);
```

- *crea un area di memoria per mappare un file a partire da un indirizzo specificato, con livello di protezione indicato*
- *da usare solo se sapete veramente cosa state facendo!!!! Soprattutto brk*
 - Studiarle sul man (o sul libro APUE*)
 - `$ man 2 mmap`
 - `$ man 2 brk`

*Advanced Programming In the Unix Environment. 3rd Edition



Allocazione della memoria (cont.)

- Le funzioni ritornano un puntatore (`void *`)
- Serve fare il casting al tipo di puntatore relativo al tipo di dato contenuto nella memoria per poter utilizzare correttamente l'aritmetica dei puntatori
- Ad esempio:

```
char *strPtr = NULL;  
const int SIZE_OF_ARRAY = 30;  
strPtr=(char *) calloc(SIZE_OF_ARRAY, sizeof(char));
```

```
void *realloc(void *ptr, size_t size)
```

- modifica la dimensione dell'area di memoria precedentemente allocata con `m/calloc` e puntata da `ptr` nella dimensione specificata dal valore di `size`
- Ritorna NULL in caso di errore
- ad es. (da es. precedente)

```
    strptr1=(char *) realloc(strptr, 10*SIZE_OF_ARRAY);
```

- `strptr1` **potrebbe essere diverso da** `strptr` infatti, nel caso di aumento della dimensione!
- Se `realloc` non riesce ad allargare l'area correntemente allocata e puntata da `ptr`, allora quest'ultima viene liberata e una nuova area allocata e ritornata al chiamante.

realloc (cont.)

- Il contenuto dell'area di memoria puntata da `ptr` non viene modificato
- La nuova area di memoria allocata NON viene inizializzata (se ad esempio quella originale lo era)
- In caso di errore (ad esempio non c'è memoria a sufficienza) l'area di memoria originale rimane intatta.
- Leggete il man!!

Memory leakage («perdita» di memoria)

- L'esecuzione di un programma che non gestisce correttamente la liberazione della memoria non più utilizzata, può causare un aumento del consumo della memoria del sistema. Questo può portare al fallimento del programma stesso, non riuscendo più ad allocare altra memoria da utilizzare, ed in generale può portare al deterioramento delle prestazioni e del funzionamento del sistema.
- **Usare sempre `free()` dopo che avete terminato l'uso della memoria allocata dinamicamente**

memset() e memcpy()

```
void *memset(void *s, int c, size_t n);
```

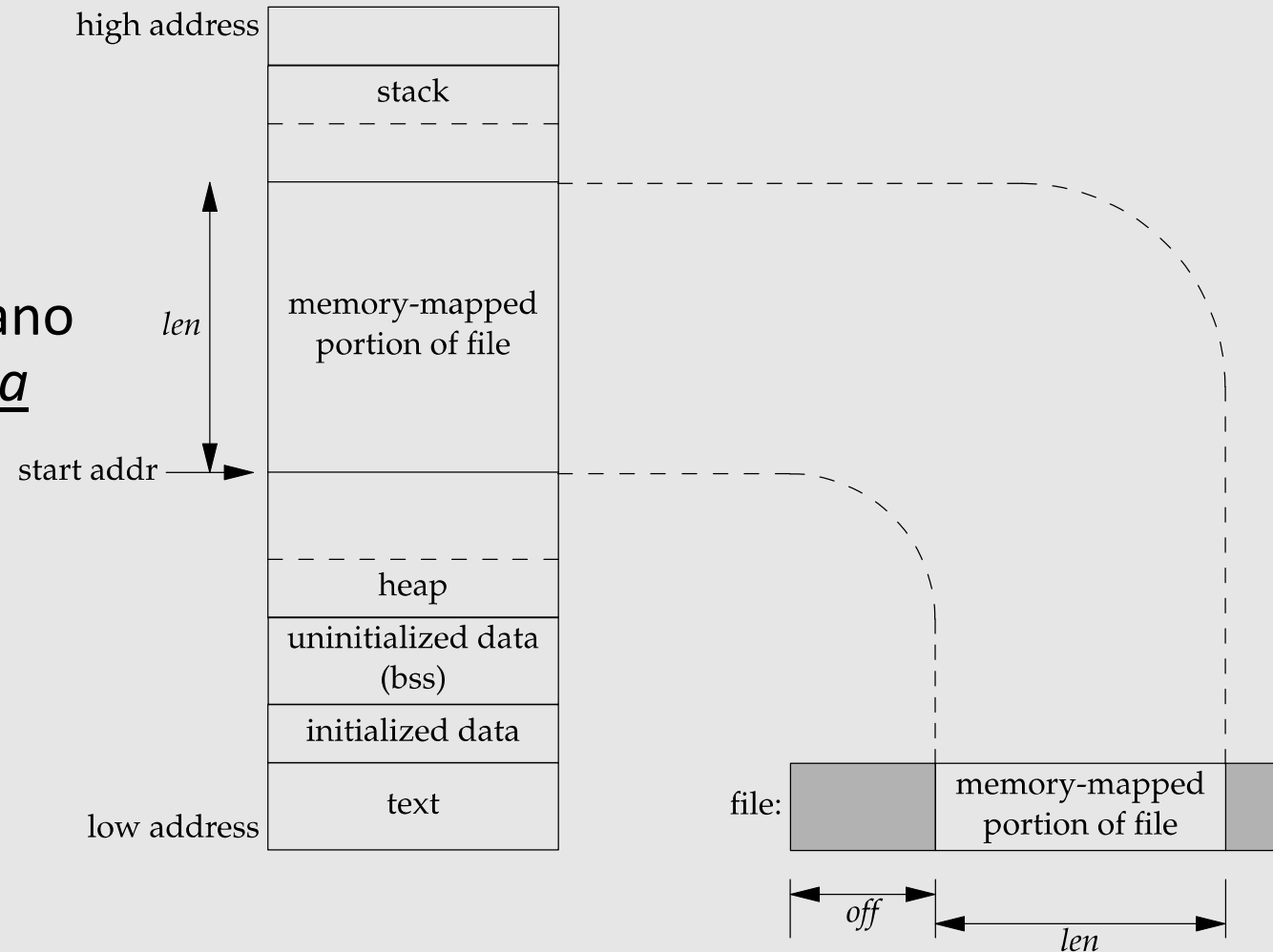
- assegna il valore intero `c` ad `n` bytes contigui dell'area di memoria puntata da `s`

```
void *memcpy(void *dest, const void *src, size_t n);
```

- copia `n` bytes contigui a partire da `src` in `dest`
 - le due aree di memoria non devono sovrapporsi
 - utile per duplicare rapidamente un array
- **ESERCIZIO:** Rifare Es.6 Lezione 9 (slide 16, copia di blocchi di memoria) usando `memcpy`. E' più semplice?!?!?
 - Vediamo soluzione Es6L9.c (file con vecchio nome!!!)

```
void *mmap (void *addr, size_t len, int
prot, int flag, int fd, off_t off )
```

- Implementa **memory-mapped I/O**
- Mappa un file su disco in un area di memoria (buffer)
- Lettura/scrittura dal/sul buffer risultano in lettura scrittura dal/sul disco (senza accedere al disco)
- Uso:
 - `#include <sys/mman.h>`
 - Restituisce l'indirizzo di partenza della regione mappata (buffer) se OK, MAP_FAILED se errore



```
void *mmap (void *addr, size_t len, int  
prot, int flag, int fd, off_t off )
```

- *addr* è l'indirizzo iniziale dell'area di memoria in cui vogliamo mappare il file. Se *addr* = NULL, il kernel sceglie per noi
- *fd* è il file descriptor (il file va aperto prima)
 - *len* il numero di byte da trasferire
 - *off* l'offset nel file
- *prot* indica il livello di protezione
- *flag*
 - MAP_SHARED le operazioni sulla memoria modificano il file (R/W)
 - MAP_PRIVATE viene creata una copia privata del mapped file e solo questa viene modificata

<i>prot</i>	Description
PROT_READ	Region can be read.
PROT_WRITE	Region can be written.
PROT_EXEC	Region can be executed.
PROT_NONE	Region cannot be accessed.

Sync e de-mapping

```
#include <sys/mman.h>
```

```
int msync(void *addr, size_t len, int flags);
```

- Le modifiche ad una regione mappata (con MAP_SHARED, ovviamente) non vengono scritte immediatamente nel file – il kernel decide quando farlo
- Con `msync` comunicate al kernel che volete scrivere sul file le modifiche in memoria

```
#include <sys/mman.h>
```

```
int munmap(void *addr, size_t len);
```

- Quando il processo termina la memoria viene automaticamente de-mappata, ma ciò non significa che il contenuto è scritto sul file (dipende dalla politica di gestione della memoria virtuale)
- Ciò non avviene se chiudo il file, quindi sempre buona regola de-mappare la memoria con `munmap`

Esempio uso `mmap` (pag. 529 di APUE)

- Lo vediamo la prossima volta perché dobbiamo introdurre altre funzioni.

